

## LAB # 07

### Getting Started with Joins, Set Operators and Views in Oracle Database

#### Objectives:

By the end of this lab, students will be able to:

- Understand the concept and purpose of SQL Joins in relational databases.
- Implement different types of joins (Inner, Outer, Cross, and Self Join) using Oracle SQL.
- Retrieve and combine data from multiple tables in the HR schema.
- Apply Set Operators (UNION, UNION ALL, INTERSECT, MINUS) to combine query results.
- Create, modify, and delete Views in Oracle SQL.

#### INTRODUCTION TO JOINS IN ORACLE

A **JOIN** is used to retrieve data from two or more tables based on a logical relationship between them.

In the Oracle **HR Schema**:

- EMPLOYEES.DEPARTMENT\_ID → references DEPARTMENTS.DEPARTMENT\_ID
  - EMPLOYEES.JOB\_ID → references JOBS.JOB\_ID
  - DEPARTMENTS.LOCATION\_ID → references LOCATIONS.LOCATION\_ID
- Joins are required because related information is stored in separate tables to maintain normalization.

#### Types of Joins:

Following are the types of joins. They are:

- Cross Join / Cartesian Join
- Inner Join / Equity Join
- Outer Join
- Left Outer
- Right Outer

- Full Outer
- Self-Join

### **CROSS JOIN (CARTESIAN JOIN)**

**Definition:** A **Cross Join** returns all possible combinations of rows from two tables.

If:

- Table A has 10 rows
- Table B has 5 rows

Result =  $10 \times 5 = 50$  rows

No condition is required.

### **Oracle Syntax (SQL-1999 Standard)**

SELECT column\_list

FROM table1

CROSS JOIN table2;

**Question:** Display all possible combinations of employee names and department names. **Query:**

SELECT E.FIRST\_NAME, D.DEPARTMENT\_NAME

FROM EMPLOYEES E

CROSS JOIN DEPARTMENTS D;

### **Output Explanation:**

Each employee will appear with every department name.

If there are 107 employees and 27 departments → result = 2889 rows.

### **INNER JOIN (EQUI JOIN)**

**Definition:** An **Inner Join** returns only matching rows from both tables based on a join condition. If there is no match, the row is excluded.

When equality (=) is used, it is called an **Equi Join**.

**Oracle Syntax**

```
SELECT column_list  
FROM table1 t1  
INNER JOIN table2 t2  
ON t1.column = t2.column;
```

**Question:** Display employee name and department name.

**Query:**

```
SELECT E.FIRST_NAME, E.LAST_NAME, D.DEPARTMENT_NAME  
FROM EMPLOYEES E  
INNER JOIN DEPARTMENTS D  
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

**Output Explanation:**

Only employees who are assigned to a department will appear.  
Employees without department\_id (NULL) will not be shown.

Columns displayed:

- FIRST\_NAME
- LAST\_NAME
- DEPARTMENT\_NAME

**NATURAL JOIN**

**Definition:** A **Natural Join** automatically joins tables based on columns with the same name.  
It joins using ALL common column names.

**Syntax:**

```
SELECT column_list  
FROM table1  
NATURAL JOIN table2;
```

**Question:** Display employee details with department name using NATURAL JOIN.

**Query:**

```
SELECT FIRST_NAME, LAST_NAME, DEPARTMENT_NAME  
FROM EMPLOYEES  
NATURAL JOIN DEPARTMENTS;
```

**Output Explanation:**

Oracle automatically joins on DEPARTMENT\_ID because it exists in both tables.

## JOIN USING CLAUSE

**Definition:** The USING clause is used when both tables contain the same column name and we want to specify that column explicitly.

**Syntax:**

```
SELECT column_list  
FROM table1  
JOIN table2  
USING (column_name);
```

**Question:** Display employee name and department location ID.

**Query:**

```
SELECT FIRST_NAME, LAST_NAME, LOCATION_ID  
FROM EMPLOYEES  
JOIN DEPARTMENTS  
USING (DEPARTMENT_ID);
```

**Output Explanation:**

Oracle joins using DEPARTMENT\_ID and displays LOCATION\_ID from DEPARTMENTS table.

## SELF JOIN

**Definition:** A **Self Join** is when a table is joined with itself.

In HR schema:

EMPLOYEES.MANAGER\_ID references

EMPLOYEES.EMPLOYEE\_ID **Syntax**

SELECT columns

FROM table t1

JOIN table t2

ON t1.column = t2.column;

**Question:** Display employee name and their manager name.

**Query:**

SELECT E.FIRST\_NAME AS EMPLOYEE,

M.FIRST\_NAME AS MANAGER

FROM EMPLOYEES E

JOIN EMPLOYEES M

ON E.MANAGER\_ID = M.EMPLOYEE\_ID;

**Output Explanation:**

- EMPLOYEE → employee name
- MANAGER → their manager name
- Employees without managers will not appear.

## OUTER JOINS

**Definition:** An **Outer Join** returns matched rows plus unmatched rows from one or both tables.

## LEFT OUTER JOIN

### Definition:

Returns:

- All rows from left table
- Matching rows from right table
- NULL for unmatched right table rows

5

FAST School of Computing

CL-2005 - Database Systems Lab Lab Manual Prepared by: Abdullah Shaikh

### Syntax:

SELECT columns

FROM table1 t1

LEFT OUTER JOIN table2 t2

ON t1.column = t2.column;

**Question:** Display all employees and their department names, including employees without departments.

### Query:

SELECT E.FIRST\_NAME, D.DEPARTMENT\_NAME

FROM EMPLOYEES E

LEFT OUTER JOIN DEPARTMENTS D

ON E.DEPARTMENT\_ID = D.DEPARTMENT\_ID;

### Output Explanation:

All employees will appear.

If an employee has no department → DEPARTMENT\_NAME will be NULL.

## RIGHT OUTER JOIN

### Definition:

Returns:

- All rows from right table
- Matching rows from left table

- NULL for unmatched left table rows

**Question:** Display all departments and their employees (including departments with no employees).

**Query:**

```
SELECT E.FIRST_NAME, D.DEPARTMENT_NAME
FROM EMPLOYEES E
RIGHT OUTER JOIN DEPARTMENTS D
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

6

FAST School of Computing

CL-2005 - Database Systems Lab

Lab Manual Prepared by: Abdullah Shaikh

**Output Explanation:**

All departments will appear.

Departments without employees will show NULL in employee column.

**FULL OUTER JOIN**

**Definition:**

Returns:

- All rows from both tables
- NULL where no match exists

**Syntax:**

```
SELECT columns
FROM table1 t1
FULL OUTER JOIN table2 t2
ON t1.column = t2.column;
```

**Question:** Display all employees and all departments, matched where possible. **Query:**

```
SELECT E.FIRST_NAME, D.DEPARTMENT_NAME
FROM EMPLOYEES E
FULL OUTER JOIN DEPARTMENTS D
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

## Output Explanation

- All employees shown
- All departments shown
- NULL where no match exists

7

FAST School of Computing

CL-2005 - Database Systems Lab

Lab Manual Prepared by: Abdullah Shaikh

## SET OPERATORS IN ORACLE

Set operators combine results of multiple SELECT statements.

### Rules:

- Same number of columns
- Same datatype order
- ORDER BY only at end

## UNION

**Definition:** Combines results of two queries and removes duplicates.

### Syntax:

SELECT columns FROM table1

UNION

SELECT columns FROM table2;

**Question:** Display all employee IDs from EMPLOYEES and JOB\_HISTORY.

### Query:

SELECT EMPLOYEE\_ID FROM EMPLOYEES

UNION



```
SELECT EMPLOYEE_ID FROM JOB_HISTORY;
```

**Output Explanation:**

All unique employee IDs from both tables.

## UNION ALL

**Definition:** Same as UNION but keeps duplicates.

**Query:**

```
SELECT EMPLOYEE_ID FROM EMPLOYEES
```

```
UNION ALL
```

```
SELECT EMPLOYEE_ID FROM JOB_HISTORY;
```

**Output Explanation:**

Duplicates are included.

## INTERSECT

**Definition:** Returns common rows between two queries.

**Question:** Display employees who exist in both EMPLOYEES and JOB\_HISTORY.

**Query:**

```
SELECT EMPLOYEE_ID FROM EMPLOYEES
```

```
INTERSECT
```

```
SELECT EMPLOYEE_ID FROM JOB_HISTORY;
```

**Output Explanation:**

Only employees who changed jobs.

## MINUS

**Definition:** Returns rows in first query but not in second.

**Question:** Display employees who never changed their job.

**Query:**

```
SELECT EMPLOYEE_ID FROM EMPLOYEES
```

```
MINUS
```

```
SELECT EMPLOYEE_ID FROM JOB_HISTORY;
```

**Output Explanation:**

Employees who do not appear in JOB\_HISTORY.

## INTRODUCTION TO VIEWS

A **View** in Oracle is a database object that allows users to see data from one or more tables in the form of a virtual table. A view does not store data physically inside the database. Instead, it stores the SQL query that retrieves data from the base tables.

Whenever a user queries a view, Oracle executes the stored query and returns the result.

In simple words:

A view is a saved SQL query that behaves like a table.

Views are commonly used in real-world systems to simplify complex queries, enhance security, and restrict access to specific data.

In the Oracle HR schema, data is stored in normalized tables such as:

- EMPLOYEES
- DEPARTMENTS
- JOBS
- JOB\_HISTORY
- LOCATIONS

Instead of repeatedly writing complex joins between these tables, we can create a view and reuse it.

## PURPOSE AND IMPORTANCE OF VIEWS

Views are used for the following reasons:

### 1. Security

A view can restrict access to specific columns or rows. For example, salary information can be hidden from certain users.

### 2. Simplification of Complex Queries

If a query involves multiple joins and conditions, it can be saved as a view to simplify usage. **3. Data Abstraction**

Users do not need to know how tables are structured internally.

### 4. Logical Data Independence

If table structure changes, the view can be modified without affecting user queries.

## SIMPLE VIEW

**Definition:** A **Simple View** is created from a single table and does not include:

- GROUP BY clause
- Aggregate functions
- Joins
- DISTINCT

Simple views generally support INSERT, UPDATE, and DELETE operations if they follow Oracle rules.

### Syntax (Oracle)

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

**Question:** Create a view that displays employee ID, first name, salary, and department ID from the EMPLOYEES table.

**Query:**

```
CREATE VIEW emp_basic_view AS
SELECT EMPLOYEE_ID,
FIRST_NAME,
SALARY,
DEPARTMENT_ID
FROM EMPLOYEES;
```

**Using the View:**

```
SELECT * FROM emp_basic_view;
```

**Output Explanation:**

The output will display four columns:

- EMPLOYEE\_ID
- FIRST\_NAME
- SALARY
- DEPARTMENT\_ID

The data comes directly from the EMPLOYEES table but appears as if it is stored in a separate table.

## VIEW WITH WHERE CONDITION

A view can restrict rows using a WHERE clause.

This is useful when we want to display only selected records.

**Question:** Create a view to display employees who earn more than 10,000 salary. **Query:**

```
CREATE VIEW high_salary_emp AS
SELECT EMPLOYEE_ID,
FIRST_NAME,
SALARY
FROM EMPLOYEES
WHERE SALARY > 10000;
```

**Using the View:**

```
SELECT * FROM high_salary_emp;
```

**Output Explanation:**

Only employees whose salary is greater than 10,000 will be displayed.

**COMPLEX VIEW**

**Definition:** A **Complex View** is created using:

- Multiple tables
- Joins
- GROUP BY clause
- Aggregate functions

Complex views are mostly read-only and may not support DML operations.

**Syntax:**

```
CREATE VIEW view_name AS  
SELECT columns  
FROM table1  
JOIN table2  
ON condition;
```

**Question:** Create a view that shows employee full name, department name, and job title. **Query:**

```
CREATE VIEW emp_complete_view AS  
SELECT E.FIRST_NAME,  
E.LAST_NAME,  
D.DEPARTMENT_NAME,  
J.JOB_TITLE  
FROM EMPLOYEES E  
JOIN DEPARTMENTS D
```

```
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID  
JOIN JOBS J  
ON E.JOB_ID = J.JOB_ID;
```

**Using the View:**

```
SELECT * FROM emp_complete_view;
```

**Output Explanation:**

The output will display:

- Employee First Name
- Employee Last Name
- Department Name
- Job Title

This simplifies a complex join into a single reusable object.

**READ-ONLY VIEW**

**Definition:** A Read-Only View prevents users from modifying data through the view.

Users can only retrieve data using SELECT statement.

**Syntax:**

```
CREATE VIEW view_name AS  
SELECT columns  
FROM table_name  
WITH READ ONLY;
```

**Question:** Create a read-only view for department details.

**Query:**

```
CREATE VIEW dept_readonly_view AS  
SELECT DEPARTMENT_ID,  
       DEPARTMENT_NAME,  
       LOCATION_ID  
FROM DEPARTMENTS
```

WITH READ ONLY;

If someone tries:

```
UPDATE dept_readonly_view  
SET DEPARTMENT_NAME = 'IT'  
WHERE DEPARTMENT_ID = 10;
```

Oracle will generate an error.

## VIEW WITH CHECK OPTION

**Definition:** The CHECK OPTION ensures that DML operations follow the condition specified in the view.

If someone tries to insert or update data that does not satisfy the view condition, Oracle rejects it.

**Question:** Create a view for employees in department 50 only, and prevent insertion of employees from other departments.

**Query:**

```
CREATE VIEW dept50_view AS  
SELECT EMPLOYEE_ID,  
FIRST_NAME,  
DEPARTMENT_ID  
FROM EMPLOYEES  
WHERE DEPARTMENT_ID = 50  
WITH CHECK OPTION;
```

If someone tries to insert:

```
INSERT INTO dept50_view  
VALUES (999, 'Ali', 60);
```

Oracle will not allow it because department\_id is not 50.

## CREATE OR REPLACE VIEW

**Definition:** To modify an existing view without dropping it, Oracle provides CREATE OR REPLACE option.

**Syntax:**

```
CREATE OR REPLACE VIEW view_name AS  
SELECT columns  
FROM table_name;
```

**Example:**

```
CREATE OR REPLACE VIEW emp_basic_view AS  
SELECT EMPLOYEE_ID,  
FIRST_NAME,  
LAST_NAME,  
SALARY  
FROM EMPLOYEES;
```

The view definition is updated.

## DROPPING A VIEW

To remove a view permanently:

```
DROP VIEW view_name;
```

Example:

```
DROP VIEW emp_basic_view;
```

## INLINE VIEW

**Definition:** An Inline View is a subquery written inside the FROM clause. It is not stored permanently in the database.



**Question:** Display employees who earn more than the average salary.

**Query:**

```
SELECT *  
FROM (  
  SELECT EMPLOYEE_ID, FIRST_NAME, SALARY  
  FROM EMPLOYEES  
)  
WHERE SALARY >  
(SELECT AVG(SALARY) FROM EMPLOYEES);
```

This temporary subquery acts like a view.

### **MATERIALIZED VIEW (Conceptual Overview)**

A Materialized View:

- Stores data physically
- Improves performance
- Used in Data Warehousing
- Requires refresh

**Syntax:**

```
CREATE MATERIALIZED VIEW view_name  
AS  
SELECT columns  
FROM table_name;
```

Unlike normal views, materialized views occupy storage space.

### **LIMITATIONS OF VIEWS**

- Complex views are not always updatable
- Performance depends on base tables
- If base table is dropped, view becomes invalid

Lab Tasks:

Q1. Display the employee first name, last name, and department name for all employees using an INNER JOIN.

Q2. Display employee first name, last name, and job title by joining the EMPLOYEES and JOBS tables.

Q3. Display employee name and department location ID using the USING clause.

Q4. Display employee name and department name using a NATURAL JOIN between EMPLOYEES and DEPARTMENTS.

- Q5. Display employee name and manager name using a SELF JOIN on the EMPLOYEES table.
- Q6. Display all employees along with their department names, including employees who do not belong to any department.
- Q7. Display all departments and their employees, including departments without employees.
- Q8. Display all possible combinations of employee names and department names.
- Q9. Display employee IDs from EMPLOYEES and JOB\_HISTORY without duplicates using a set operator.
- Q10. Display employee IDs that exist in both EMPLOYEES and JOB\_HISTORY tables.
- Q11. Display employee name, department name, and city by joining EMPLOYEES, DEPARTMENTS, and LOCATIONS tables.
- Q12. Create a view named emp\_dept\_view that displays:
- EMPLOYEE\_ID
  - FIRST\_NAME
  - LAST\_NAME
  - DEPARTMENT\_NAME
- by joining EMPLOYEES and DEPARTMENTS tables.
- Q13. Create a view that displays employees whose salary is greater than the average salary of the company using an inline view.
- Q14. Display employees who never changed their job using EMPLOYEES and JOB\_HISTORY tables with a set operator.

**THE END!**